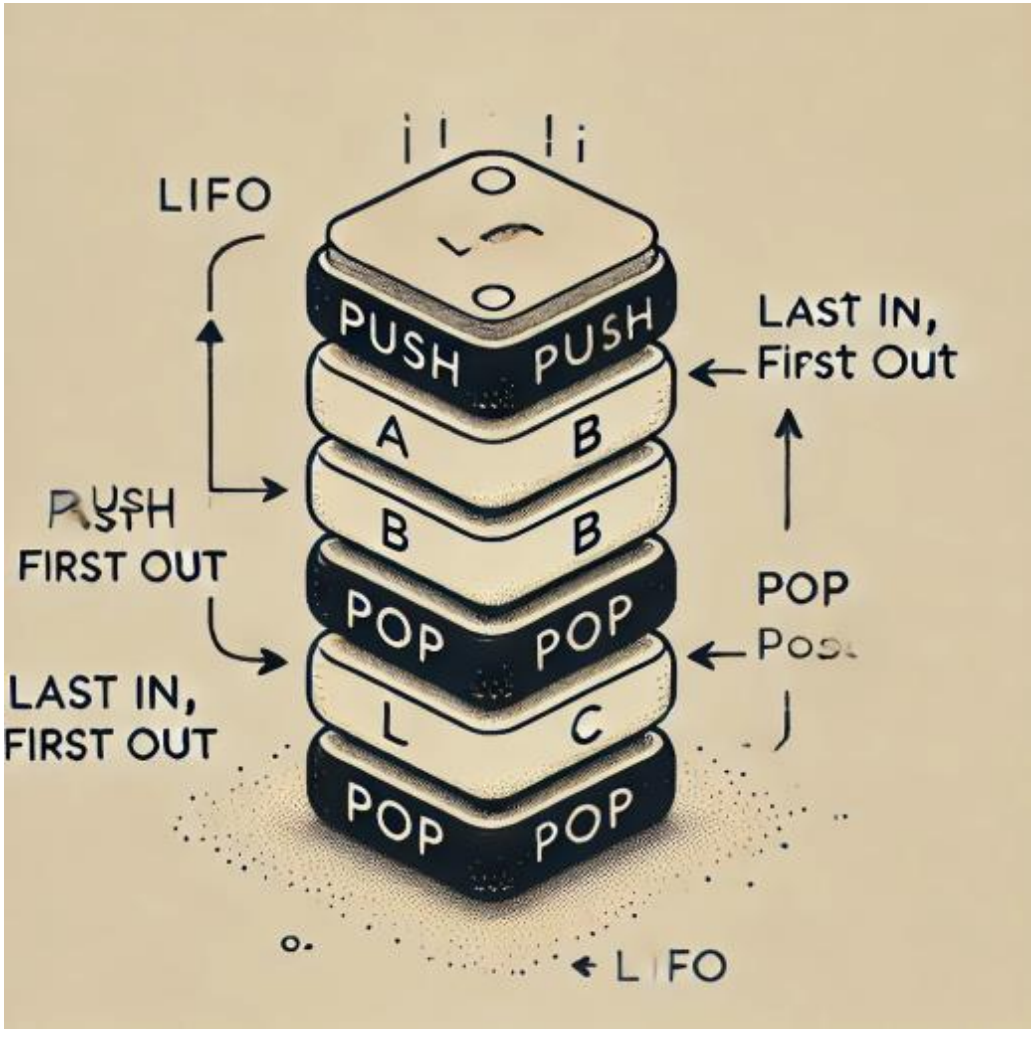
Stacks


LIFO
PUSH PUSH
A B
B B
POP POP
L C
POP POP
LAST IN,
First Out
POP
FIRST OUT
LAST IN,
FIRST OUT
LIFO

Definition

Stack: Abstract data type with the following operations:

Definition

Stack: Abstract data type with the following operations:

- `Push(Key)`: adds key to collection

Definition

Stack: Abstract data type with the following operations:

- `Push(Key)`: adds key to collection
- `Key Top()`: returns most recently-added key

Definition

Stack: Abstract data type with the following operations:

- `Push(Key)`: adds key to collection
- `Key Top()`: returns most recently-added key
- `Key Pop()`: removes and returns most recently-added key

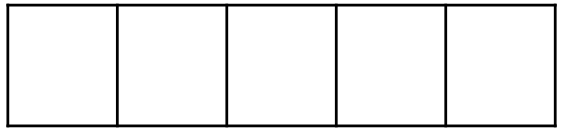
Definition

Stack: Abstract data type with the following operations:

- `Push(Key)`: adds key to collection
- `Key Top()`: returns most recently-added key
- `Key Pop()`: removes and returns most recently-added key
- `Boolean Empty()`: are there any elements?

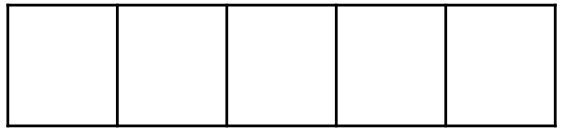
Stack Implementation with Array

numElements: 0



Stack Implementation with Array

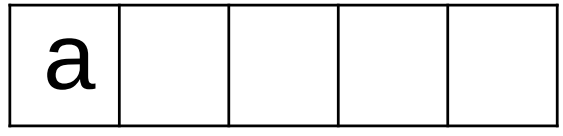
numElements: 0



`Push(a)`

Stack Implementation with Array

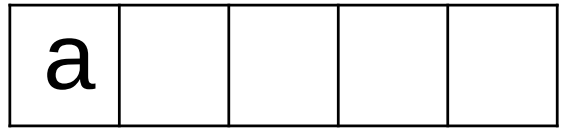
numElements: 1



`Push(a)`

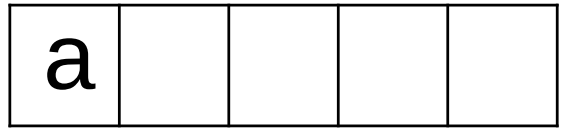
Stack Implementation with Array

numElements: 1



Stack Implementation with Array

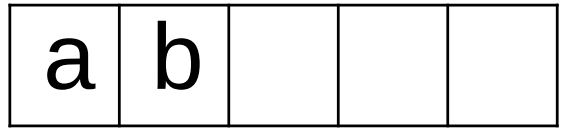
numElements: 1



`Push(b)`

Stack Implementation with Array

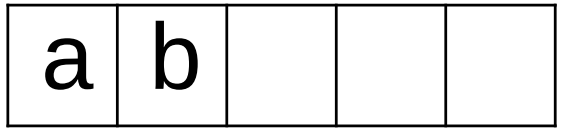
numElements: 2



`Push(b)`

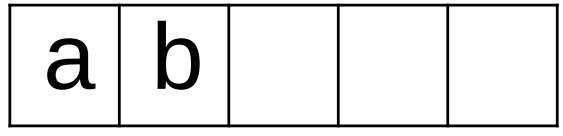
Stack Implementation with Array

numElements: 2



Stack Implementation with Array

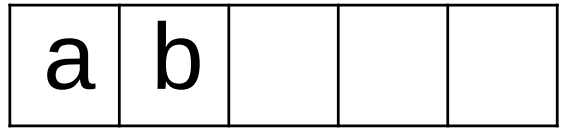
numElements: 2



`Top()`

Stack Implementation with Array

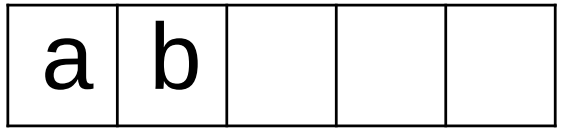
numElements: 2



`Top()` → `b`

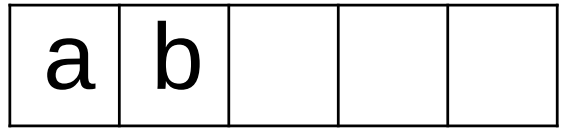
Stack Implementation with Array

numElements: 2



Stack Implementation with Array

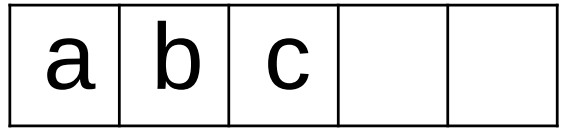
numElements: 2



`Push(c)`

Stack Implementation with Array

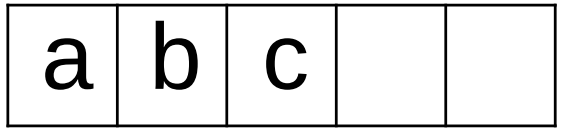
numElements: 3



`Push(c)`

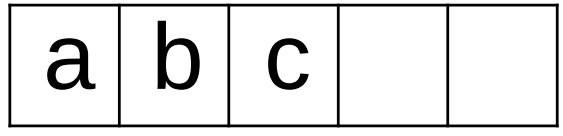
Stack Implementation with Array

numElements: 3



Stack Implementation with Array

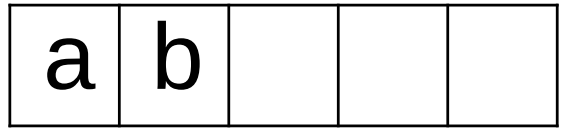
numElements: 3



`Pop()`

Stack Implementation with Array

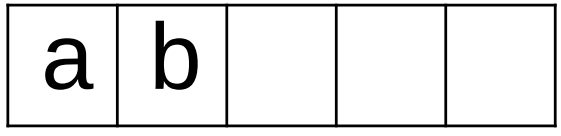
numElements: 2



`Pop()` → `c`

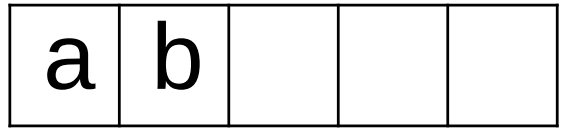
Stack Implementation with Array

numElements: 2



Stack Implementation with Array

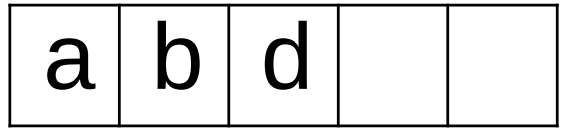
numElements: 2



`Push(d)`

Stack Implementation with Array

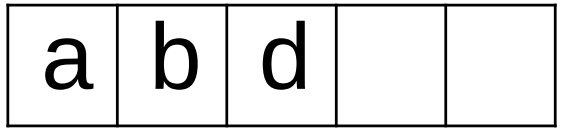
numElements: 3



`Push(d)`

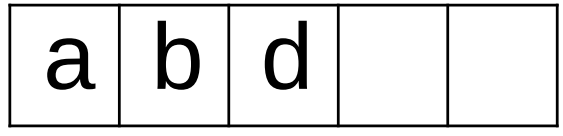
Stack Implementation with Array

numElements: 3



Stack Implementation with Array

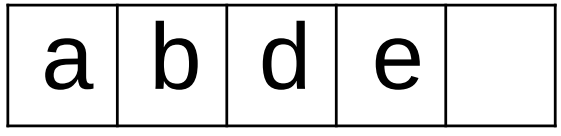
numElements: 3



`Push(e)`

Stack Implementation with Array

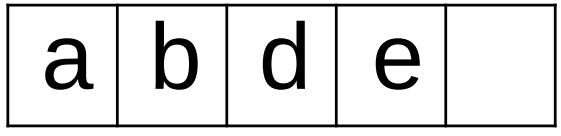
numElements: 4



`Push(e)`

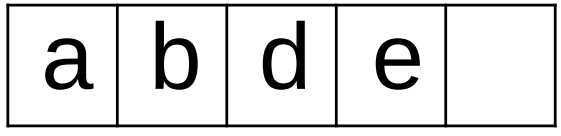
Stack Implementation with Array

numElements: 4



Stack Implementation with Array

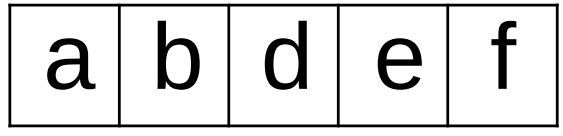
numElements: 4



`Push(f)`

Stack Implementation with Array

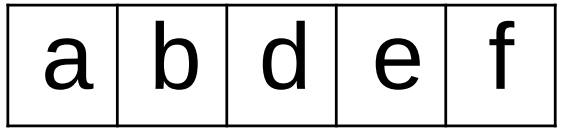
numElements: 5



`Push(f)`

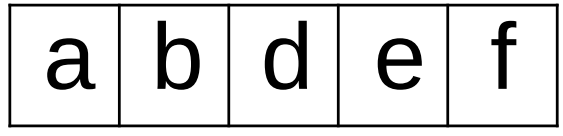
Stack Implementation with Array

numElements: 5



Stack Implementation with Array

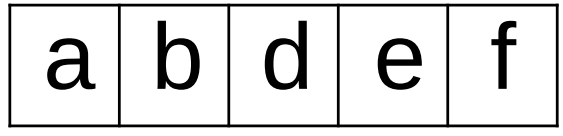
numElements: 5



`Push(g)`

Stack Implementation with Array

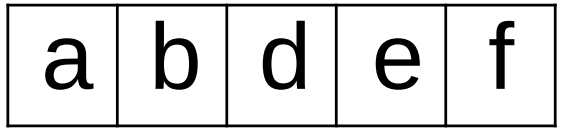
numElements: 5



`Push(g)` → `ERROR`

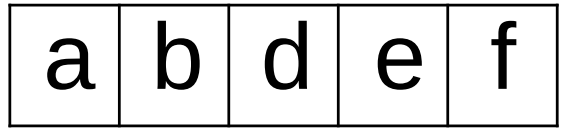
Stack Implementation with Array

numElements: 5



Stack Implementation with Array

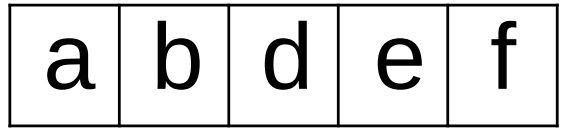
numElements: 5



`Empty()`

Stack Implementation with Array

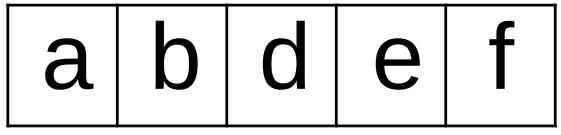
numElements: 5



`Empty()` → `False`

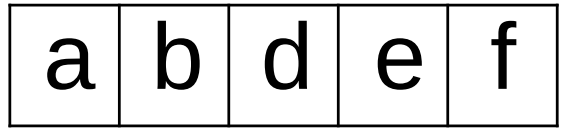
Stack Implementation with Array

numElements: 5



Stack Implementation with Array

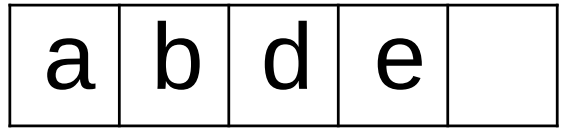
numElements: 5



Pop ()

Stack Implementation with Array

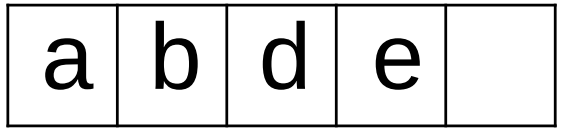
numElements: 4



`Pop()` → `f`

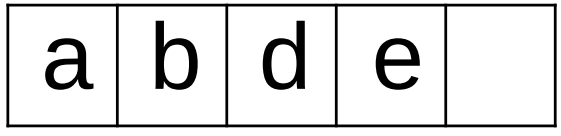
Stack Implementation with Array

numElements: 4



Stack Implementation with Array

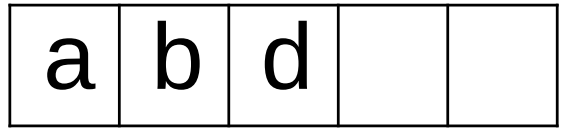
numElements: 4



Pop ()

Stack Implementation with Array

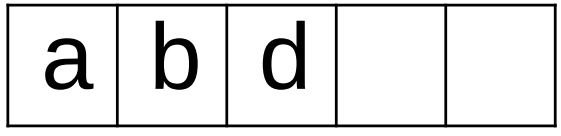
numElements: 3



Pop () → e

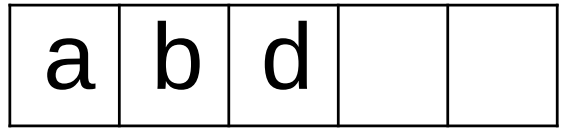
Stack Implementation with Array

numElements: 3



Stack Implementation with Array

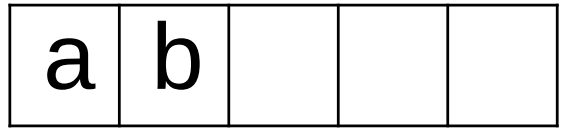
numElements: 3



Pop ()

Stack Implementation with Array

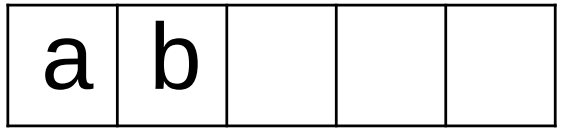
numElements: 2



`Pop()` → `d`

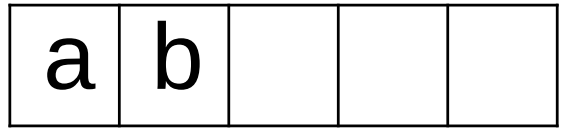
Stack Implementation with Array

numElements: 2



Stack Implementation with Array

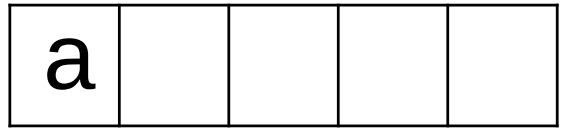
numElements: 2



Pop ()

Stack Implementation with Array

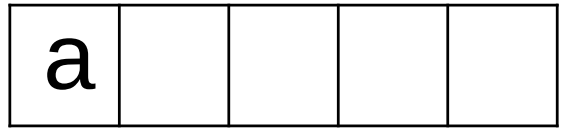
numElements: 1



`Pop()` → `b`

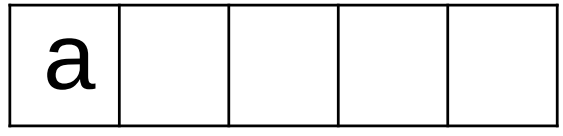
Stack Implementation with Array

numElements: 1



Stack Implementation with Array

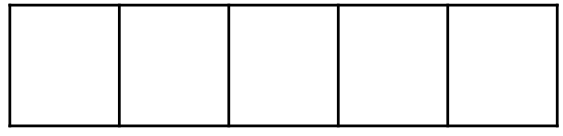
numElements: 1



Pop ()

Stack Implementation with Array

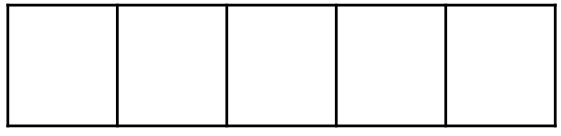
numElements: 0



`Pop()` → `a`

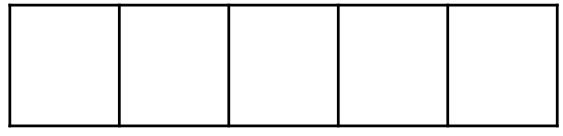
Stack Implementation with Array

numElements: 0



Stack Implementation with Array

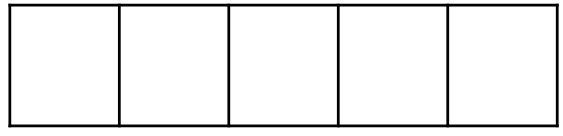
numElements: 0



`Empty()`

Stack Implementation with Array

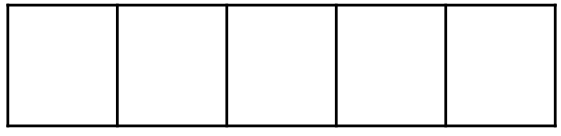
numElements: 0



`Empty()` → `True`

Stack Implementation with Array

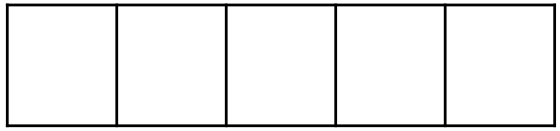
numElements: 0



Question

Consider the array below and do the following operation

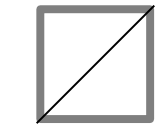
Push (a)
Push (b)
Top ()
Push (c)
Pop ()
Push (d)
Push (e)
Push (f)
Push (g)
Pop ()
Push (g)
Empty ()



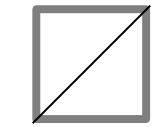
Question

Illustrate the result of each operation in the sequence PUSH(S, 4), PUSH(S, 1), PUSH(S, 3), POP(S), PUSH(S, 8), and POP(S) on an initially empty stack S stored in array S[1.. 6]

Stack Implementation with Linked List

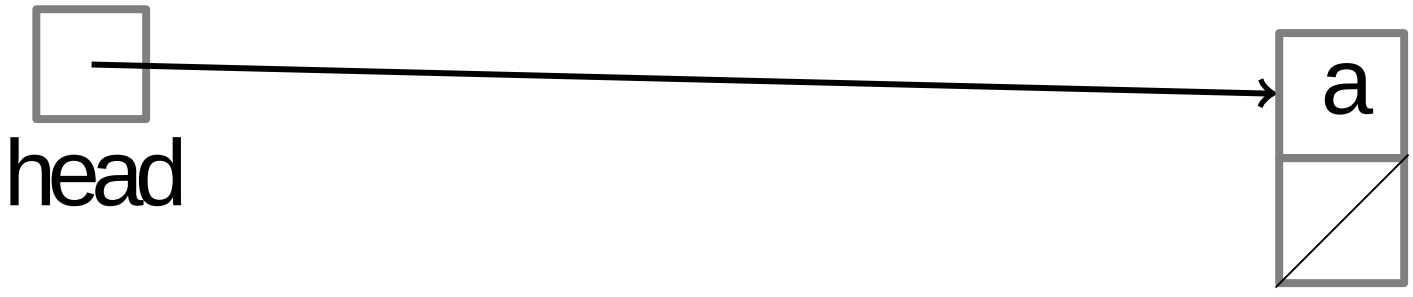


Stack Implementation with Linked List



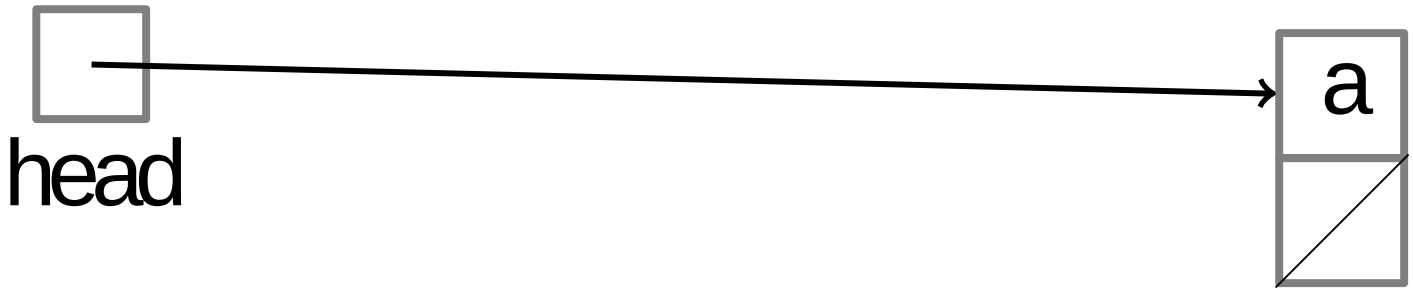
`Push(a)`

Stack Implementation with Linked List

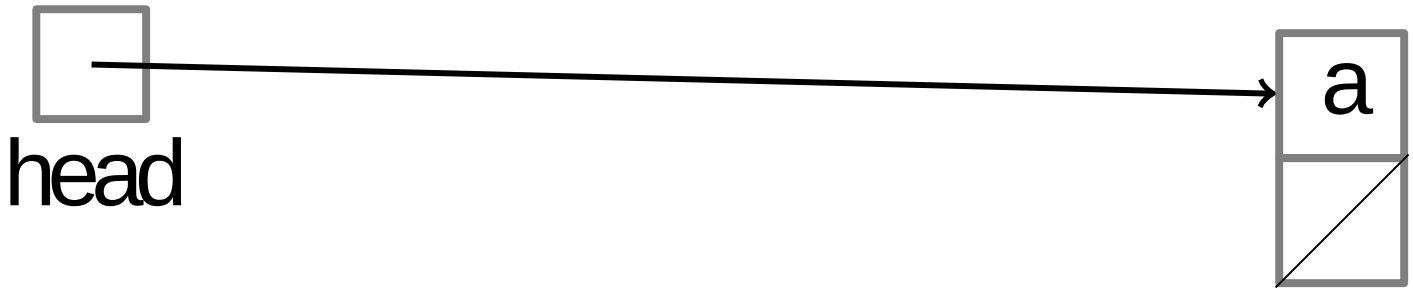


`Push(a)`

Stack Implementation with Linked List

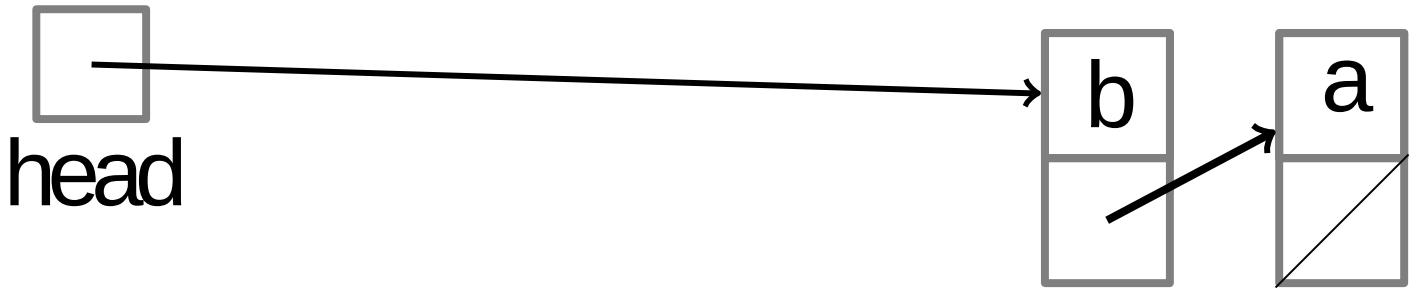


Stack Implementation with Linked List



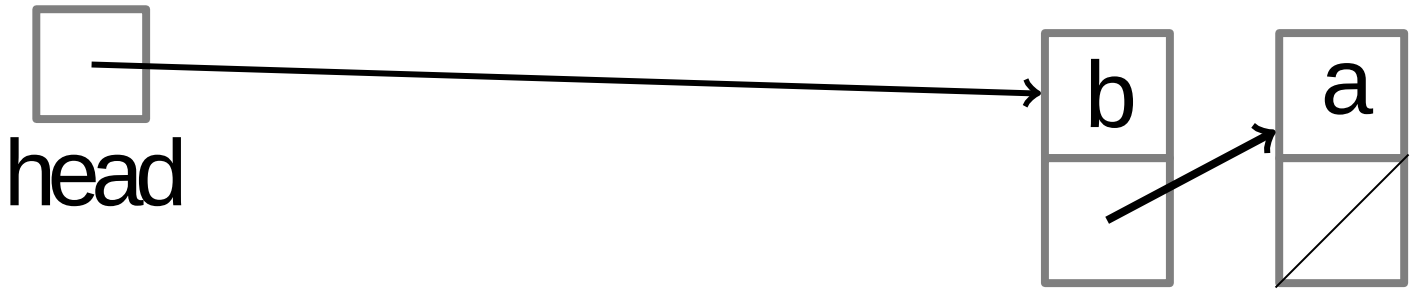
`Push(b)`

Stack Implementation with Linked List

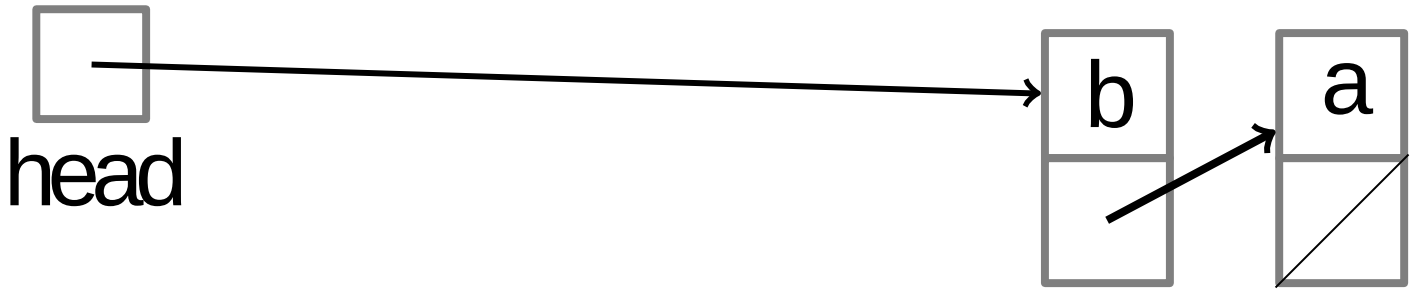


`Push(b)`

Stack Implementation with Linked List

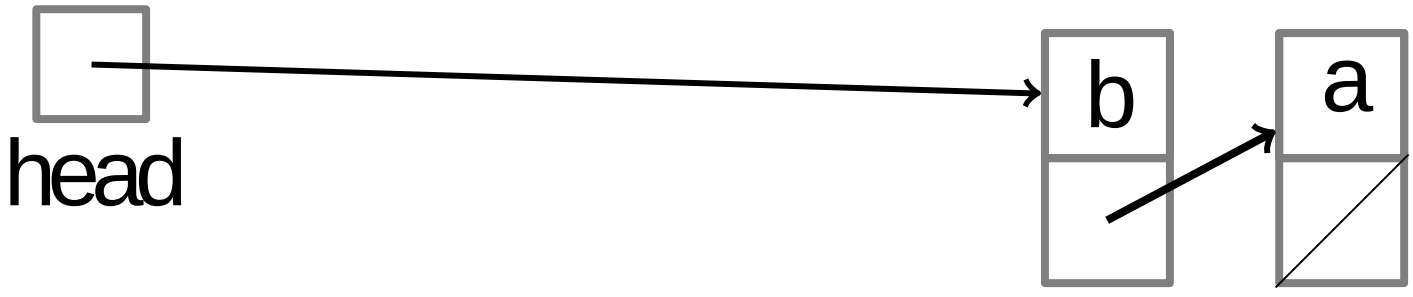


Stack Implementation with Linked List



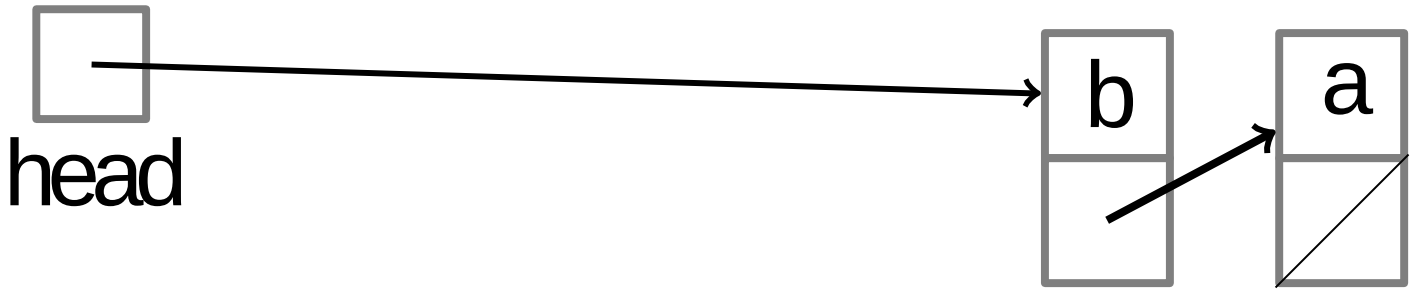
`Top()`

Stack Implementation with Linked List

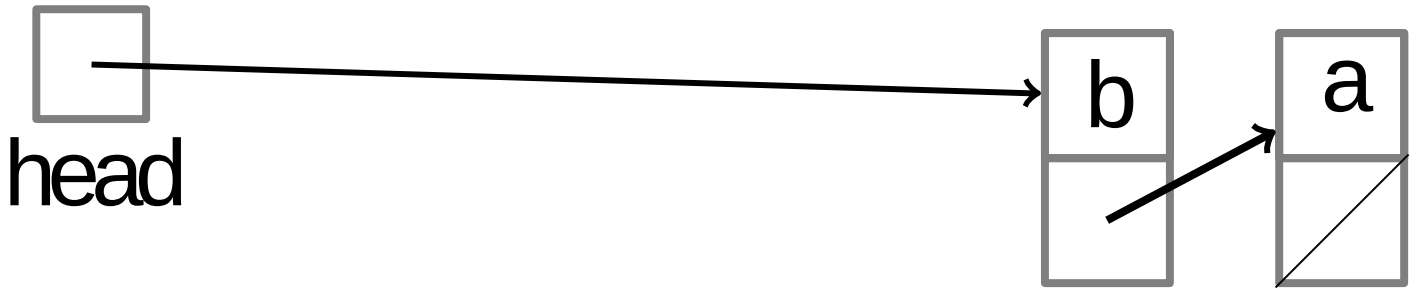


`Top()` → `b`

Stack Implementation with Linked List

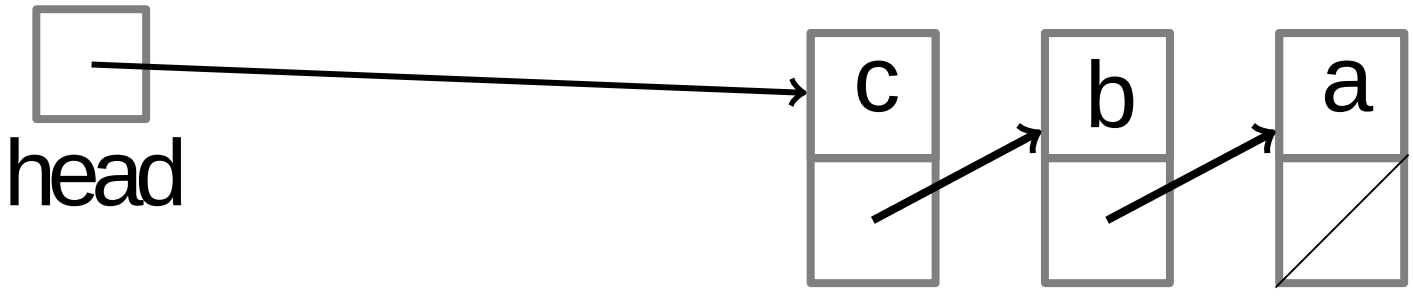


Stack Implementation with Linked List



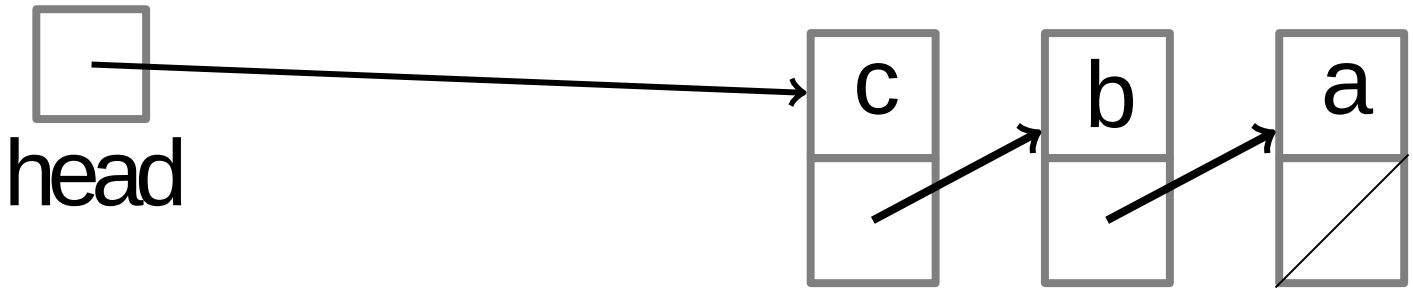
`Push(c)`

Stack Implementation with Linked List

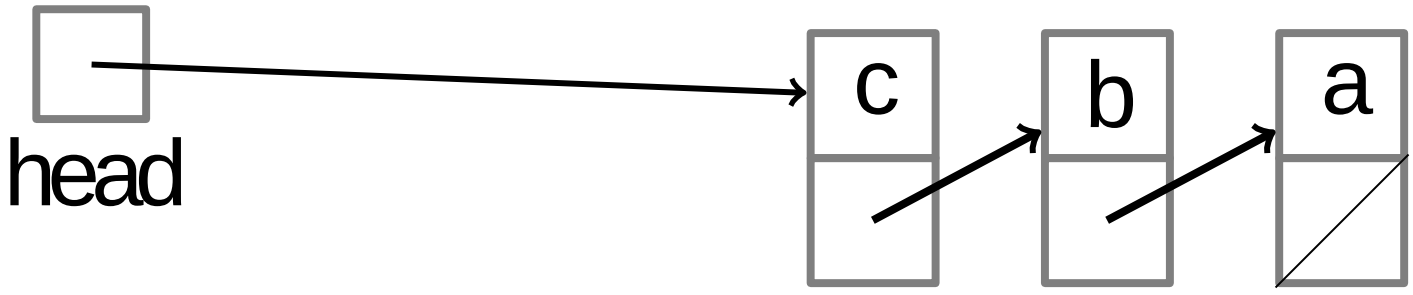


`Push(c)`

Stack Implementation with Linked List

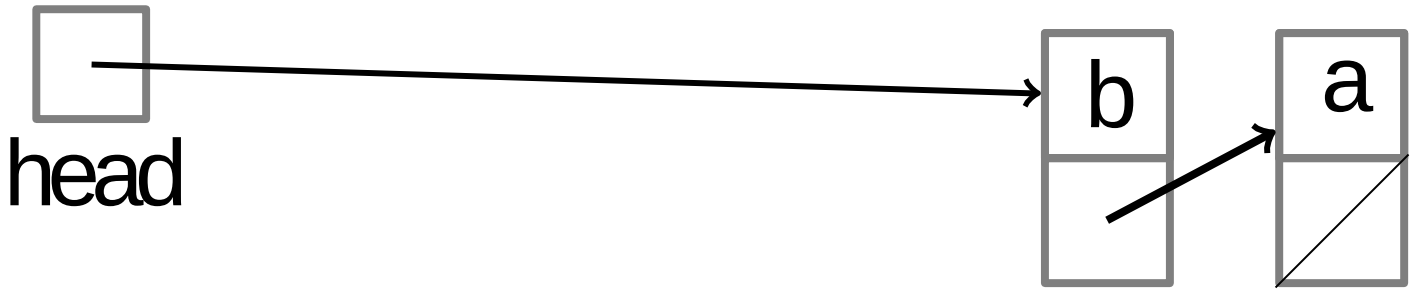


Stack Implementation with Linked List



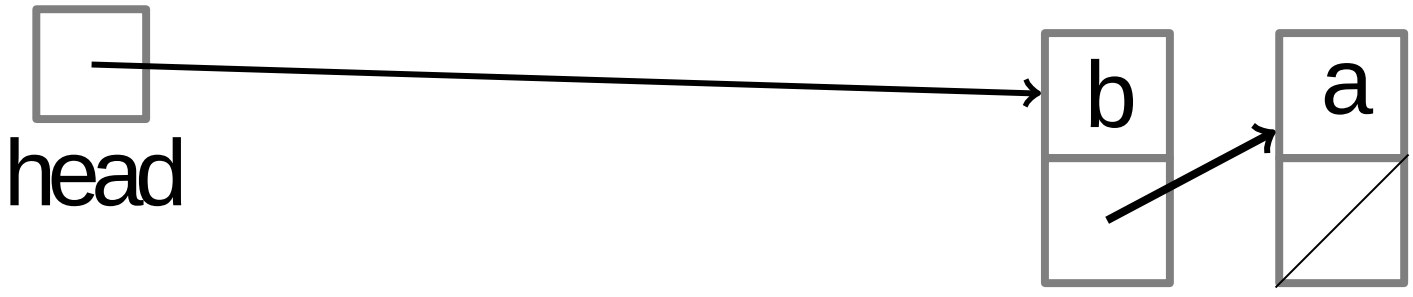
Pop()

Stack Implementation with Linked List

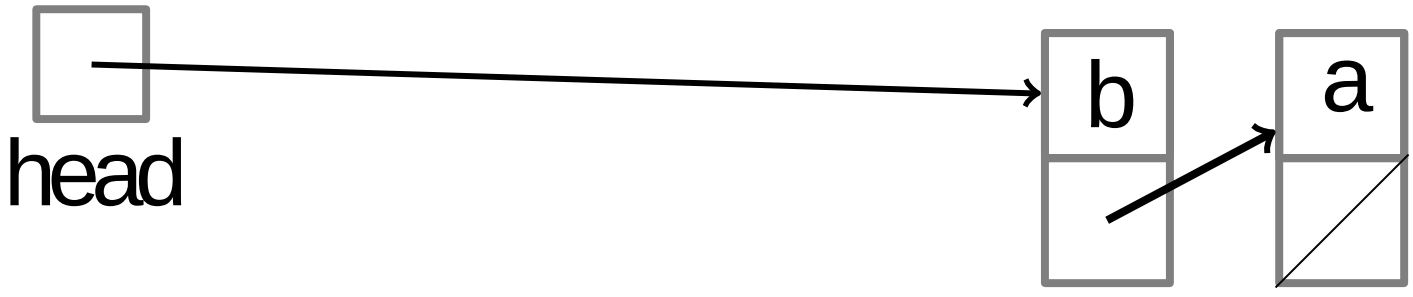


`Pop()` → `c`

Stack Implementation with Linked List

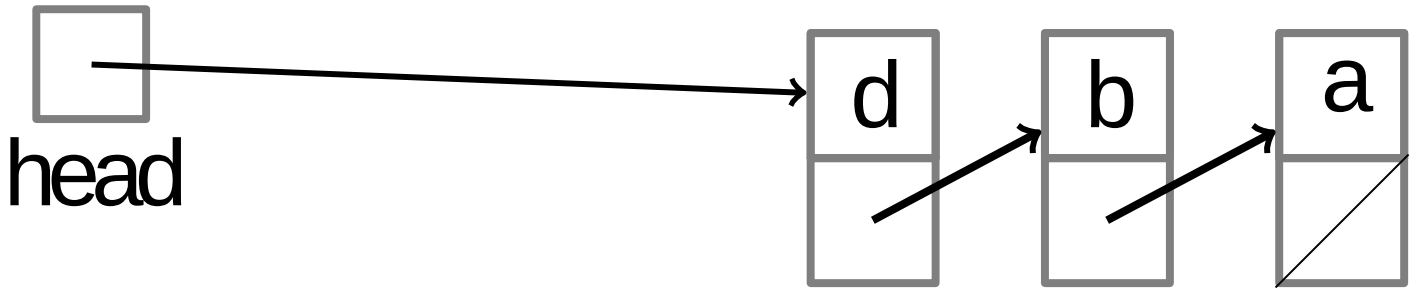


Stack Implementation with Linked List



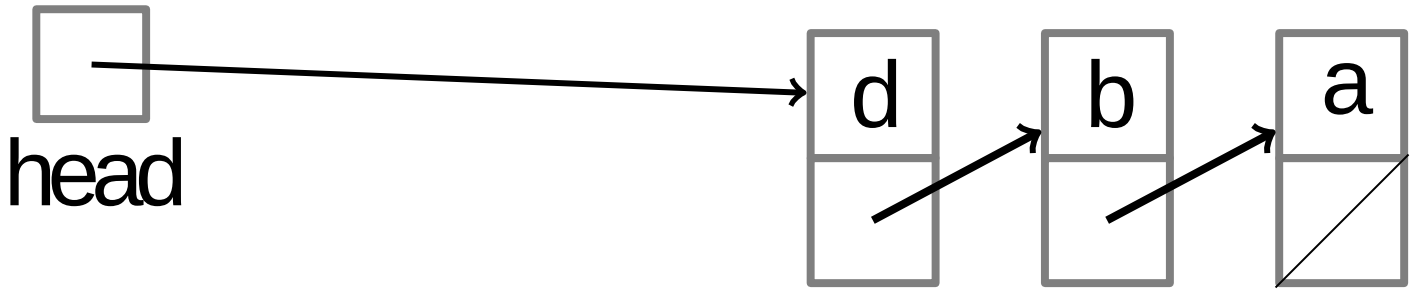
`Push(d)`

Stack Implementation with Linked List

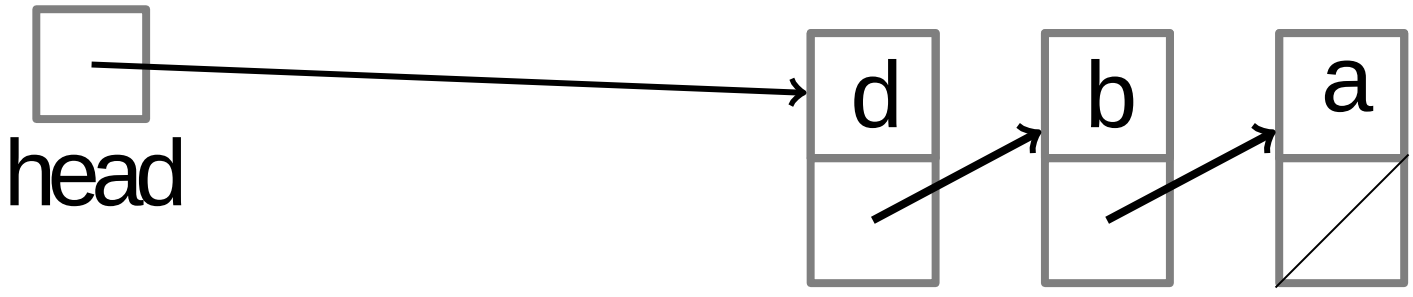


`Push(d)`

Stack Implementation with Linked List

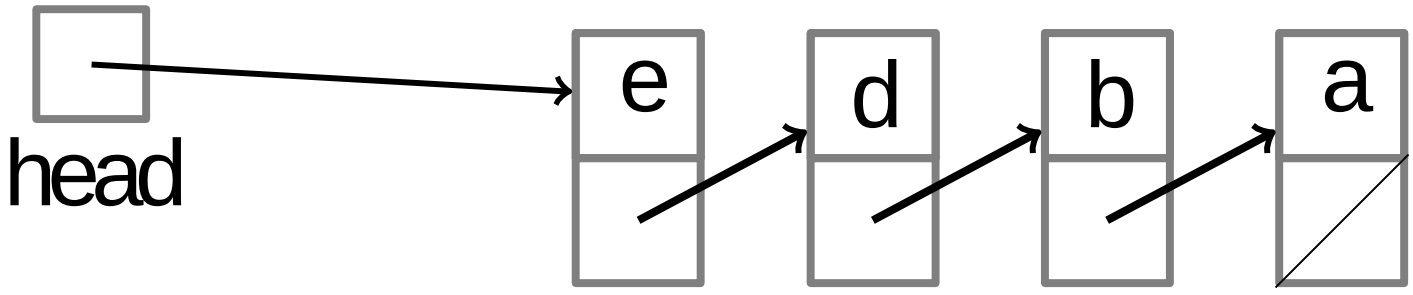


Stack Implementation with Linked List



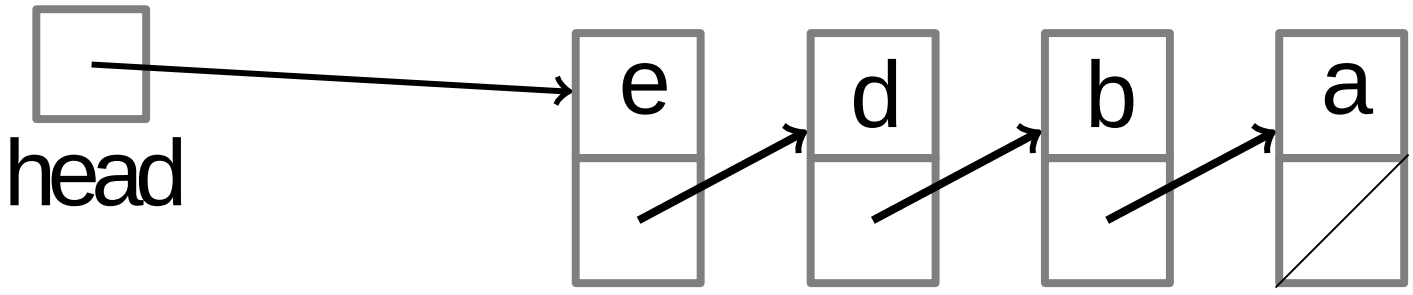
`Push(e)`

Stack Implementation with Linked List

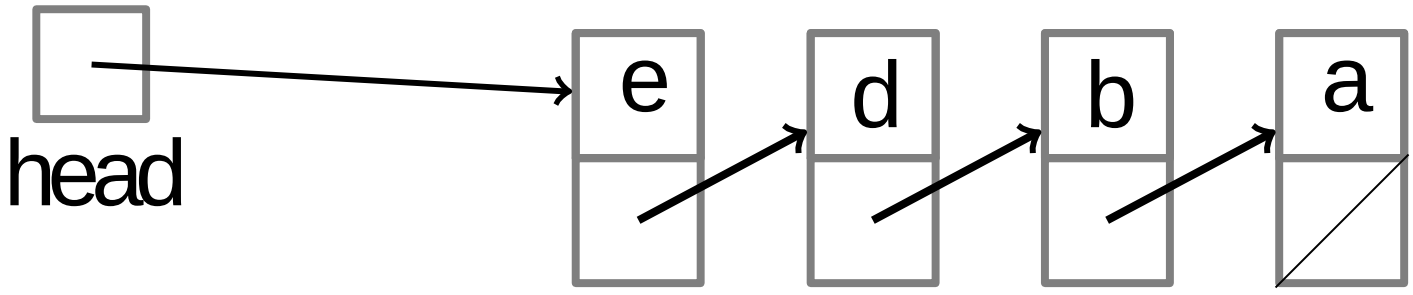


`Push(e)`

Stack Implementation with Linked List

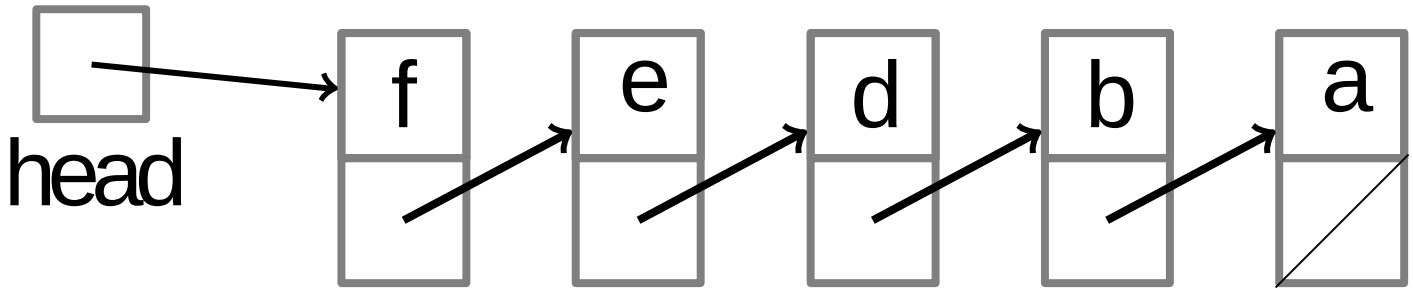


Stack Implementation with Linked List



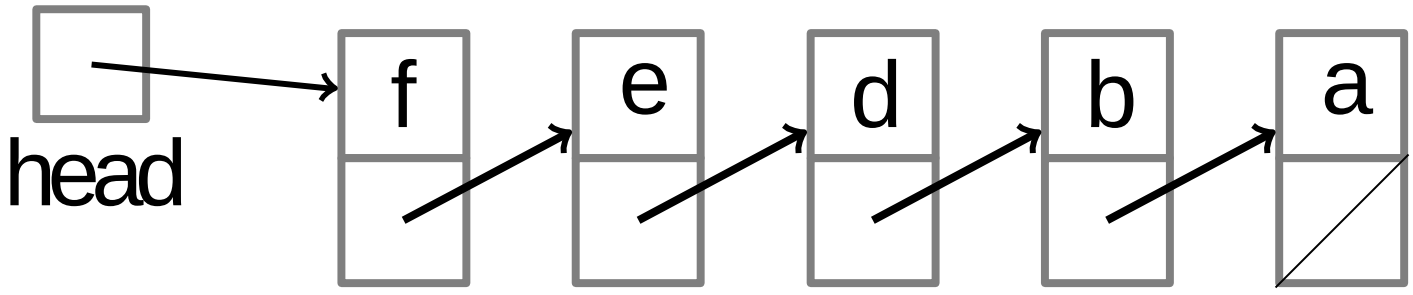
`Push(f)`

Stack Implementation with Linked List

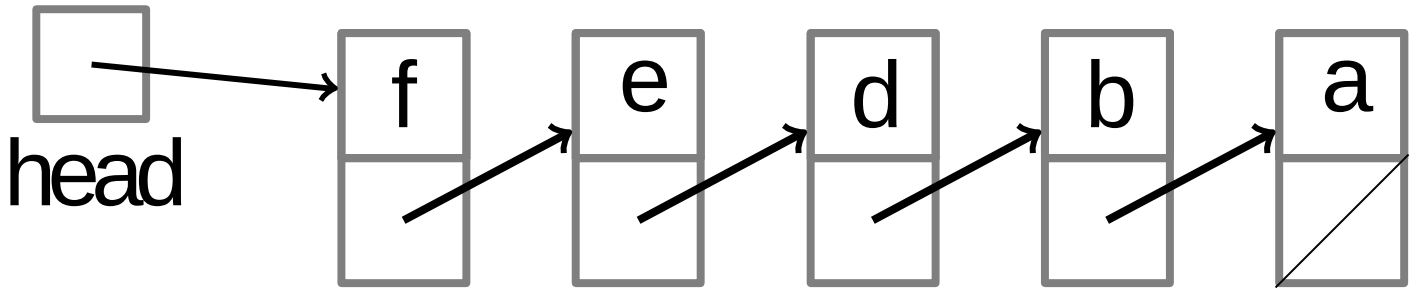


`Push(f)`

Stack Implementation with Linked List

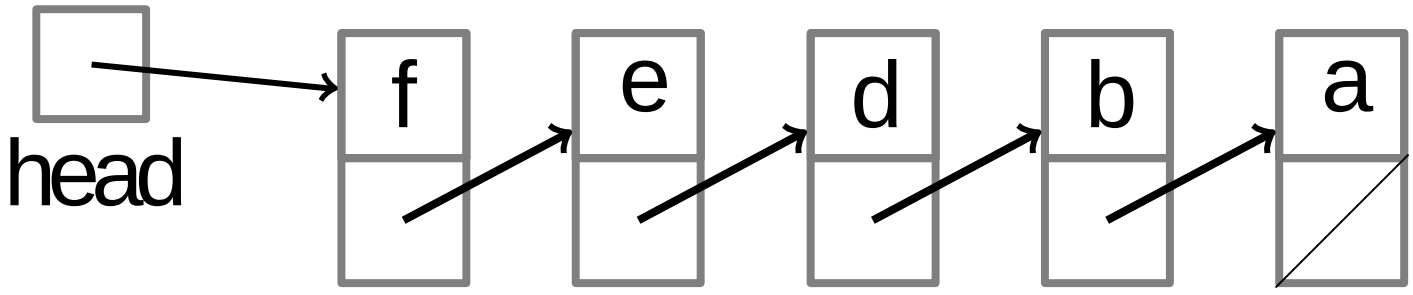


Stack Implementation with Linked List



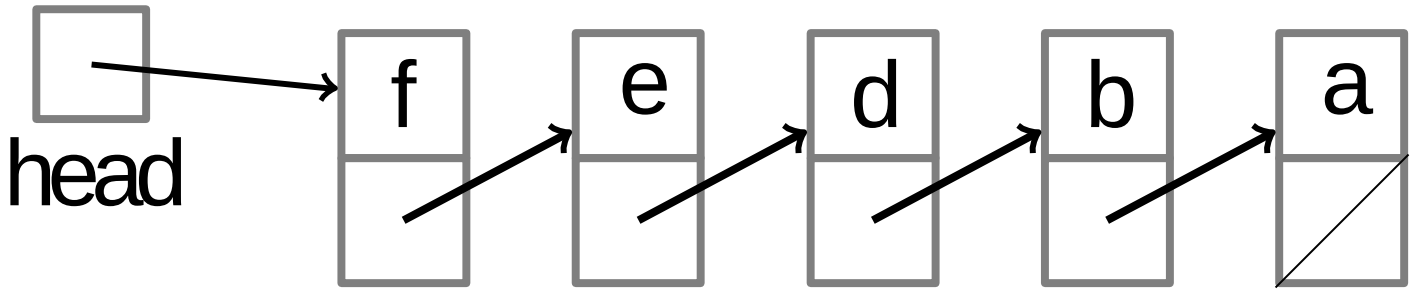

```
Empty()
```

Stack Implementation with Linked List

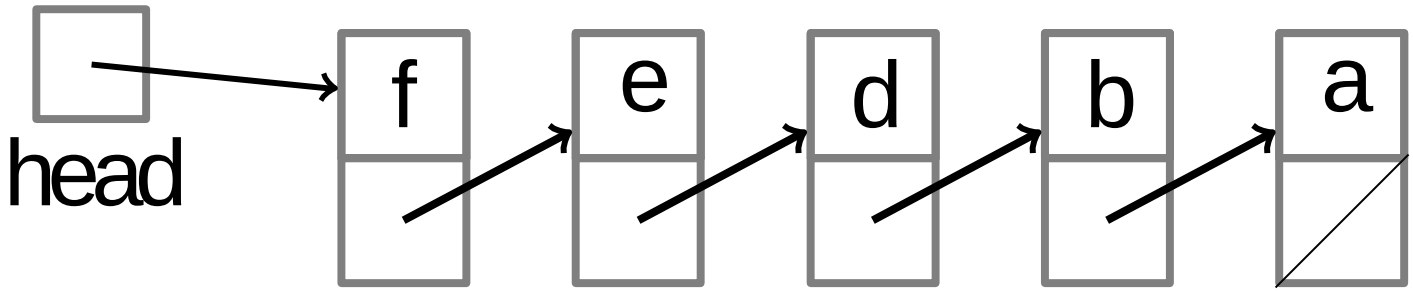


`Empty()` → `False`

Stack Implementation with Linked List

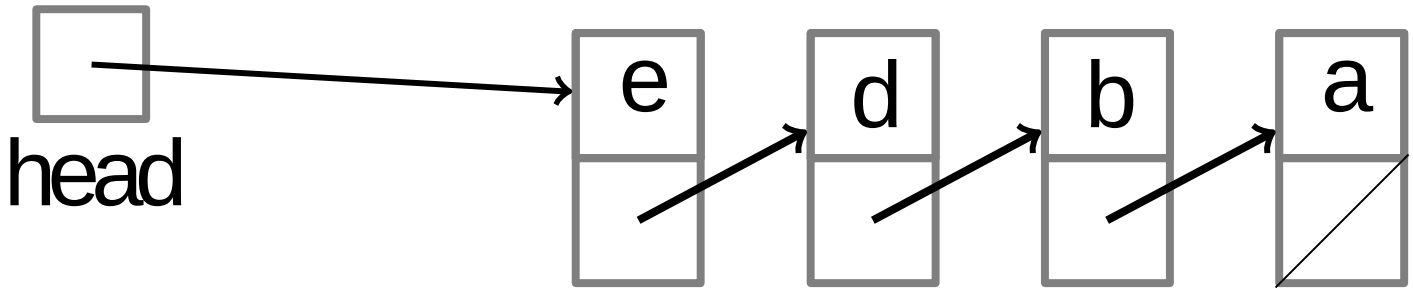


Stack Implementation with Linked List



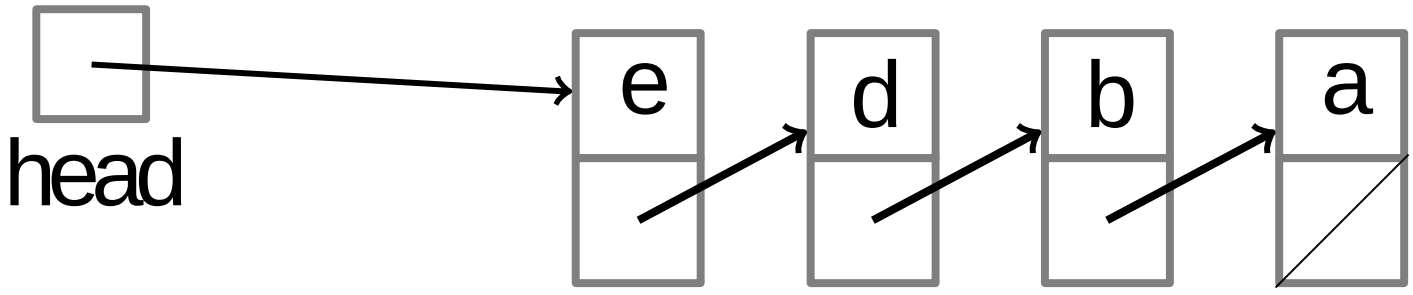
Pop()

Stack Implementation with Linked List

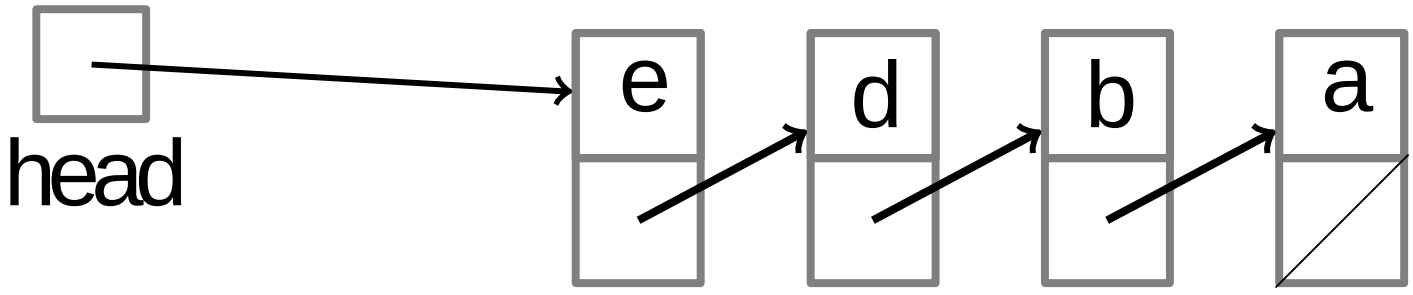


Pop()→ f

Stack Implementation with Linked List

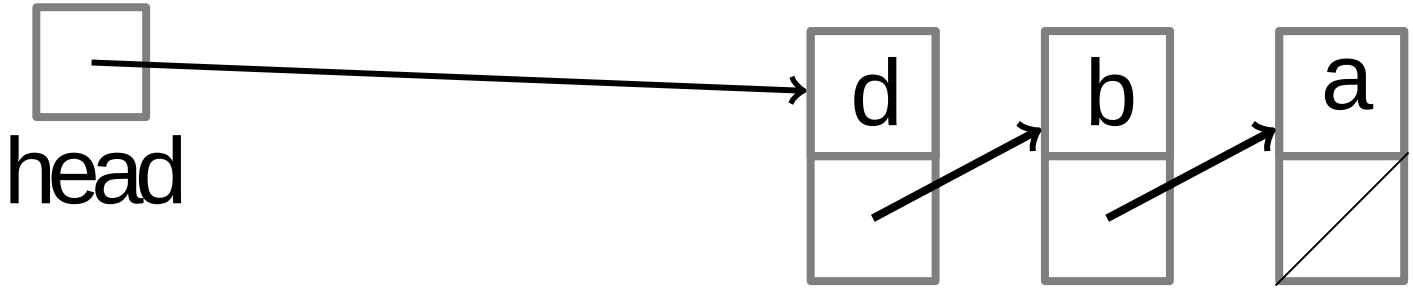


Stack Implementation with Linked List



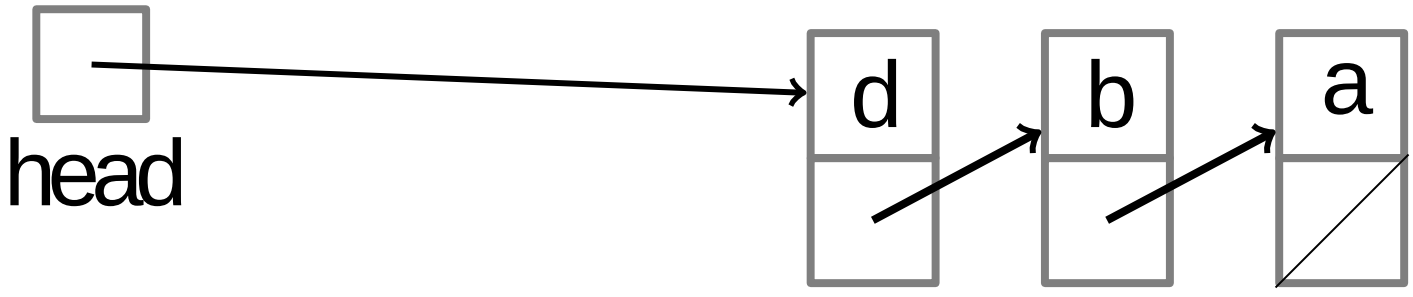
Pop()

Stack Implementation with Linked List

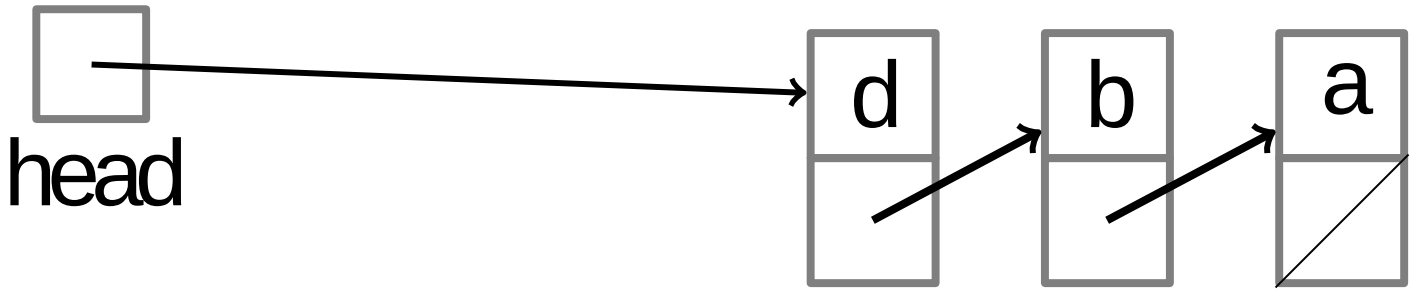


`Pop()` → `e`

Stack Implementation with Linked List

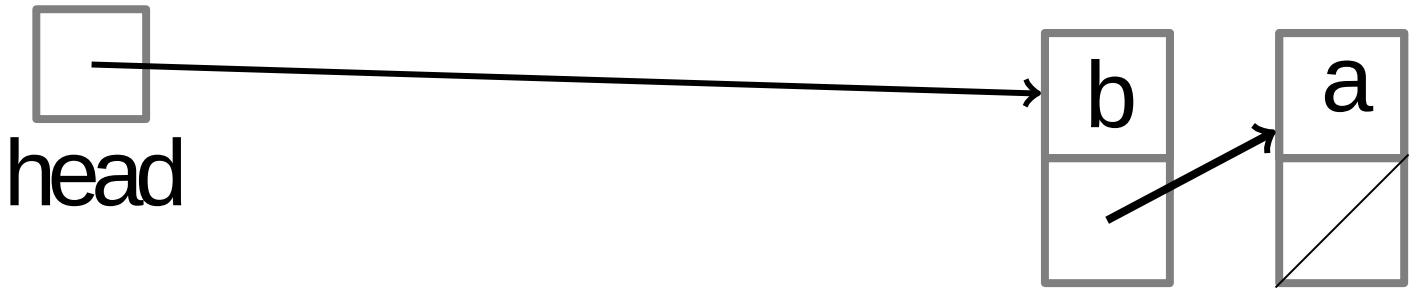


Stack Implementation with Linked List



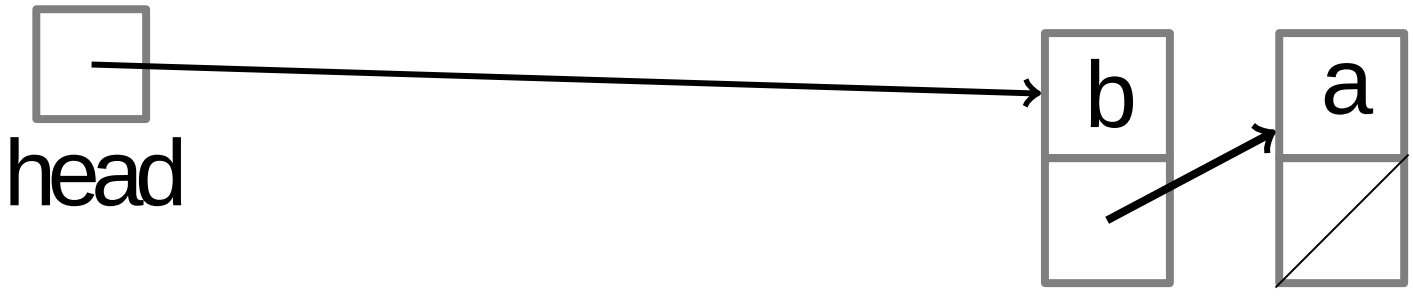
Pop()

Stack Implementation with Linked List

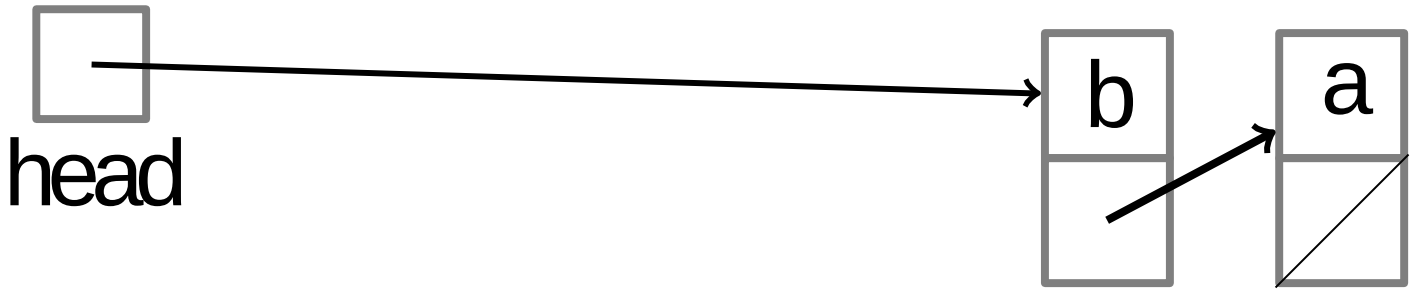


`Pop()` → `d`

Stack Implementation with Linked List

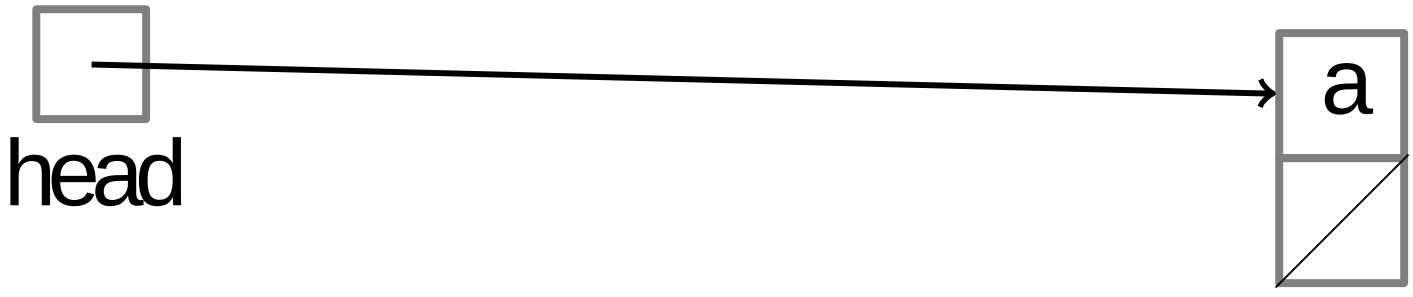


Stack Implementation with Linked List



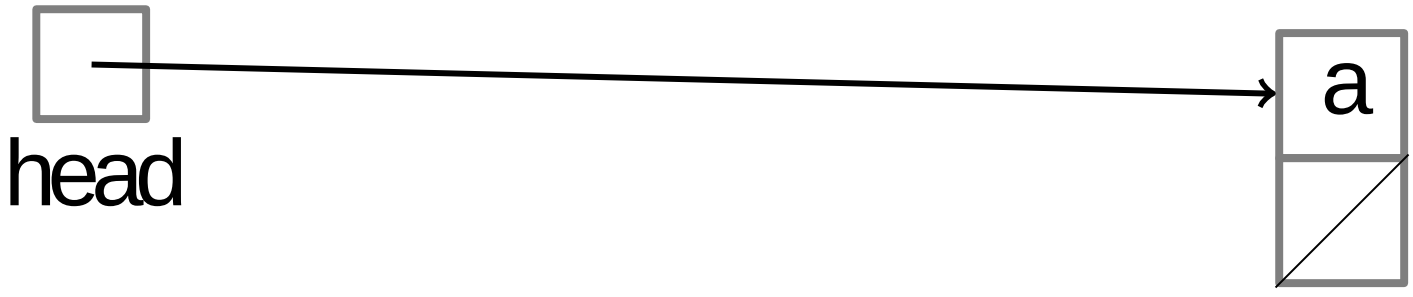
Pop()

Stack Implementation with Linked List

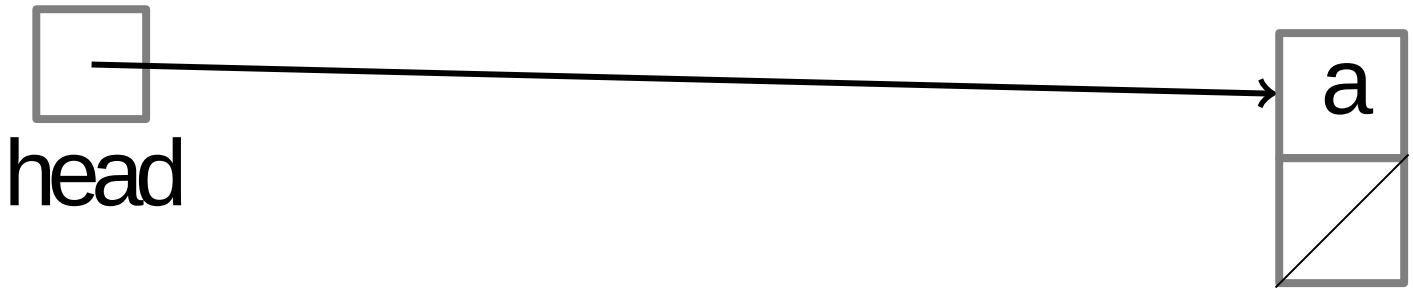


`Pop()` → `b`

Stack Implementation with Linked List

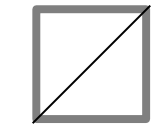


Stack Implementation with Linked List



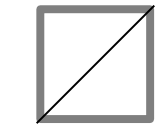
Pop()

Stack Implementation with Linked List

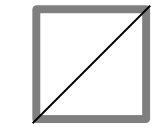


`Pop()` → `a`

Stack Implementation with Linked List

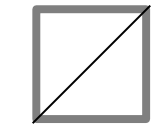


Stack Implementation with Linked List



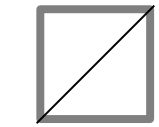
`Empty()`

Stack Implementation with Linked List



`Empty()` → `True`

Stack Implementation with Linked List



Stack Implementation with Linked List

Stack Implementation with Linked List

Stack Implementation with Linked List

Summary

- Stacks can be implemented with either an array or a linked list.

Summary

- Stacks can be implemented with either an array or a linked list.
- Each stack operation is $O(1)$: Push, Pop, Top, Empty.

Summary

- Stacks can be implemented with either an array or a linked list.
- Each stack operation is $O(1)$: Push, Pop, Top, Empty.
- Stacks are ocassionaly known as LIFO queues.

Last In First Out

Question 1: Stack Implementation Using Array

```
class ArrayStack:
    def __init__(self, capacity):
        self.stack = [None] * capacity
        self.top_index = -1  // Represents an empty stack

    def Push(key):
        if top_index == capacity - 1:
            print("Stack Overflow")
        else:
            top_index += 1
            stack[top_index] = key

    def Pop():
        if top_index == -1:
            print("Stack Underflow")
            return None
        else:
            key = stack[top_index]
            top_index -= 1
            return key

    def Top():
        if top_index == -1:
            print("Stack is Empty")
            return None
        else:
            return stack[top_index]
```

Question 1: Stack Implementation Using Array

Questions:

1. Using the given pseudocode, analyze the following:

- What will happen when you attempt to Pop() an element from an empty stack?
- If the array has a capacity of 5 and you attempt to push 6 elements, what would happen? Justify your answer based on the pseudocode.

2. Determine the time complexity of the following operations:

- Push operation
- Pop operation
- Top operation

3. Explain the limitations of using an array for stack implementation, especially in terms of memory allocation.

Question 2: Stack Implementation Using Queue

```
class Node:
    def __init__(self, key):
        self.key = key
        self.next = None

class LinkedListStack:
    def __init__(self):
        self.top = None  // Empty stack is represented by a None top

    def Push(key):
        new_node = Node(key)
        new_node.next = self.top
        self.top = new_node

    def Pop():
        if self.top is None:
            print("Stack Underflow")
            return None
        else:
            key = self.top.key
            self.top = self.top.next
            return key

    def Top():
        if self.top is None:
            print("Stack is Empty")
            return None
        else:
            return self.top.key
```

Question 2: Stack Implementation Using Queue

Questions:

1. Using the given pseudocode, analyze the following:

- What happens when the stack is empty and you attempt to Pop an element?
- What is the space complexity of this stack implementation for n elements, considering both data and pointers?

2. Determine the time complexity of the following operations:
- Push operation
- Pop operation
- Top operation

3. Compare this linked list-based implementation with the array-based implementation in terms of:

- Memory usage
- Flexibility (in terms of stack size)
- Performance under different load conditions.

Question 3: Comparative Analysis

Scenario: Suppose you have a system that must handle a large and dynamic number of elements, and the system cannot predict the maximum number of elements in advance.

Question:

Which stack implementation (Array or Linked List) would you choose for this system? Justify your answer based on memory usage, time complexity, and the need for dynamic sizing.

Next Class Queue